



Cyberscope

A *TAC Security* Company

Audit Report

MagnumMeta

May 2026

Network BSC

Addresses

0x48a2728Db6854f2844c2F77423Ad75876d7c60F7

Audited by

Table of Contents

Table of Contents	1
Risk Classification	3
Review	4
Overview	5
Magnummeta Validator	5
Magnummeta Affiliate	5
MagnummetaValidatorNFT	6
Findings Breakdown	7
Diagnostics	8
IPED - Inflated Principal Enables Drain	9
Description	9
Recommendation	9
ESIR - Epoch Snapping Inflates Rewards	10
Description	10
Recommendation	11
TSI - Tokens Sufficiency Insurance	12
Description	12
Recommendation	12
Team Update	13
TTSR - Transfer-Tax Token Swap Risk	14
Description	14
Recommendation	14
Team Update	14
UPS - Untrusted Pricing Source	16
Description	16
Recommendation	16
Team Update	17
UBAF - Upgrade Bypasses APR Floor	18
Description	18
Recommendation	18
CCR - Contract Centralization Risk	20
Description	20
Recommendation	22
Team Update	22
EWRE - Epoch Window Restricts Exits	23
Description	23
Recommendation	23

Team Update	24
MEE - Missing Events Emission	25
Description	25
Recommendation	25
Team Update	25
MPV - Missing Parameter Validation	26
Description	26
Recommendation	27
Team Update	27
PF - Pausable Functionality	28
Description	28
Recommendation	29
Team Update	29
UTPD - Unverified Third Party Dependencies	30
Description	30
Recommendation	30
Team Update	30
Functions Analysis	31
Inheritance Graph	35
Flow graph	36
Summary	37
Disclaimer	38
About Cyberscope	39

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Explorer

<https://bscscan.com/address/0x48a2728db6854f2844c2f77423ad75876d7c60f7>

Audit Updates**Initial Audit**

20 Apr 2026

Corrected Phase 2

08 May 2026

Corrected Phase 3

18 May 2026

Source Files**Filename**

SHA256

MagnummetaValidator.sol

3e82465a73d7414d50db19ed218ba9a1cfba445b500b
42e9b5fa4edb0f4940a3

Overview

The MagnumMeta system is a three-contract staking suite built around a validator/delegator product, an affiliate referral program, and an NFT-based validator identity layer.

MagnummetaValidator.sol is the economic core: it accepts USDT deposits, splits them across token buying, affiliate payouts, and protocol wallets, tracks validator and delegator positions, accrues MFORGE-denominated rewards over time, supports claiming, compounding, transferring, unstaking, migration, and upgrades. MagnumMetaAffiliate.sol manages a 5-level referral tree and USDT reward accounting for uplines, while MagnumMetaValidatorNFT.sol represents validator seats as transferable-by-controller NFTs whose metadata evolves by staking phase. Together, the design aims to combine fixed-tier validator access, delegator participation, multi-level referrals, and gamified NFT progression in a single on-chain product.

Magnummeta Validator

MagnummetaValidator.sol implements the main staking lifecycle and most of the protocol's business logic. Users can either buy a validator position at a configured tier or enter as a delegator with a minimum USDT deposit; in both cases the contract pulls USDT, splits it according to configurable percentages, swaps part of it into MFORGE through a DEX router, calculates a principal and protocol-matched bonus, and opens a position that accrues rewards using a phase-based APR schedule with halving behavior and a minimum APR floor. It also supports reward claiming or auto-compounding, early and late unstaking with different settlement paths, validator NFT transfers with fees, delegator-to-validator upgrades, migration of legacy positions, reward-pool deposits and withdrawals, oracle/router/token/wallet updates, pausing, tier administration, and a broad set of read helpers for frontend dashboards and account discovery.

Magnummeta Affiliate

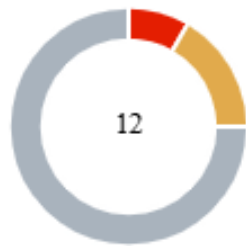
MagnumMetaAffiliate.sol implements the referral subsystem for the broader platform. When the validator contract processes a new purchase or stake, it can register the buyer's referrer on first use, provided the referrer already has an active position, and then distribute the affiliate allocation across up to five upline levels using a fixed split schedule. Rather than paying immediately, the contract credits each eligible referrer's claimable USDT balance, tracks total lifetime earnings, stores direct-downline relationships for tree traversal, and

places any undistributed remainder into a fallback balance held by the contract. It also includes legacy registration support for migrated users, owner-controlled referrer updates and token recovery, affiliate reward claiming, and several helper views for dashboards such as direct referral counts, upline trees, paginated downlines, and global affiliate-distribution statistics.

MagnummetaValidatorNFT

MagnumMetaValidatorNFT.sol is the NFT representation layer for validator positions and is tightly coupled to the validator contract. It mints one ERC721 token per validator seat, stores the validator tier associated with each token, tracks both historical mint counts and currently occupied seats per tier, and restricts minting and burning so only the validator contract can create or destroy seats. Its metadata system is dynamic: the contract stores per-tier descriptive metadata and per-tier/per-phase image URLs, then builds a fully on-chain Base64 tokenURI that reflects the validator's current staking phase by querying the validator contract. The NFT contract also intentionally blocks arbitrary peer-to-peer transfers, allowing seat movement only through the validator contract's controlled transfer flow, and it provides admin functions for maintaining tier metadata, phase images, and the authorized validator contract address.

Findings Breakdown



● Critical	1
● Medium	2
● Minor / Informative	9

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	1	0	0
● Medium	0	2	0	0
● Minor / Informative	0	9	0	0

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	IPED	Inflated Principal Enables Drain	Acknowledged
●	ESIR	Epoch Snapping Inflates Rewards	Acknowledged
●	UPS	Untrusted Pricing Source	Acknowledged
●	UBAF	Upgrade Bypasses APR Floor	Acknowledged
●	CCR	Contract Centralization Risk	Acknowledged
●	EWRE	Epoch Window Restricts Exits	Acknowledged
●	MEE	Missing Events Emission	Acknowledged
●	MPV	Missing Parameter Validation	Acknowledged
●	PF	Pausable Functionality	Acknowledged
●	TSI	Tokens Sufficiency Insurance	Acknowledged
●	TTSR	Transfer-Tax Token Swap Risk	Acknowledged
●	UTPD	Unverified Third Party Dependencies	Acknowledged

IPED - Inflated Principal Enables Drain

Criticality	Critical
Location	MagnummetaValidator.sol#L1037
Status	Acknowledged

Description

The protocol credits users with staking principal derived from the full deposited USDT amount even though only the buyPercent portion of the deposit is actually used to acquire MFORGE. This creates a mismatch between recorded staking liabilities and the actual reserves backing those positions. For example, with a 100 USDT deposit and a 1 USDT = 1 MFORGE price, only the configured buy portion is swapped into MFORGE, while the user is still credited with stake based on the full deposit amount after fees. As a result, users can accumulate staking balances that exceed the value actually acquired by the protocol. Although the immediate early-unstake drain path has been mitigated in V2, the core accounting inconsistency remains unresolved. Over time, this may cause the protocol to become undercollateralized and unable to fully honor outstanding user balances

```
function _processInvestmentAndSwap(uint256 amount) internal returns
(uint256 userPrincipalNet, uint256 affiliateAmount) {
    // Split payment: 15% Buy, 20% Affiliate, 65% DevOps
    uint256 buyAmount = (amount * buyPercent) / 100;
    affiliateAmount = (amount * affiliatePercent) / 100;
    uint256 devopsAmount = amount - buyAmount - affiliateAmount;
```

Recommendation

Consider ensuring that all credited principal, compounded balances, and redemption paths are backed by the actual amount of MFORGE acquired for each position rather than the full pre-split deposit amount. More generally, the staking, accounting, reward, migration, and withdrawal flows should operate on a single economically consistent accounting basis so that total protocol liabilities always remain aligned with real reserves held by the protocol.

ESIR - Epoch Snapping Inflates Rewards

Criticality	Medium
Location	MagnumMetaValidator.sol#L348,361,461
Status	Acknowledged

Description

The reward accounting uses the user's own `lastUpdateTime` to calculate elapsed time, but after a claim it resets that timestamp to the start of the current global epoch. Those two concepts do not always line up.

This creates problems around epoch boundaries.

If a user stakes after an epoch has already started and claims shortly after the next epoch begins, they may not have completed a full personal 7-day reward period yet. In that case, `calculateRewards()` returns no claimable reward and treats the elapsed time as pending. But `claimRewards()` still moves `lastUpdateTime` forward to `currentEpochStart`, so the user's accrued time before that epoch boundary is lost.

The reverse can also happen. A user who stakes near the end of an epoch can wait one full personal reward period, claim just before the freeze window, and then claim again when the next epoch starts. The first claim pays from the user's original timestamp, but the contract then moves `lastUpdateTime` back to the global epoch start. On the next epoch, the guard no longer blocks the user, and part of the already-paid time can be counted again.

For example, a user stakes at day 6.4 and claims at day 13.4. The contract sees 7 days elapsed and pays one epoch. It then stores `lastUpdateTime` as day 7, even though the claim happened at day 13.4. When day 14 starts, the user can claim again, and the contract calculates another full epoch from day 7 to day 14. The time from day 7 to day 13.4 was already included in the first claim, so it gets counted twice.

```
(uint256 compounded, uint256 claimable, ) = calculateRewards(positionId,
true);

validator.rewardsEarned += (compounded + claimable);
validator.lastUpdateTime = currentEpochStart;

uint256 totalTime = block.timestamp - currentTime;
uint256 totalEpochs = totalTime / EPOCH_DURATION;
uint256 pendingTime = totalTime % EPOCH_DURATION;

if (validator.lastUpdateTime >= currentEpochStart) revert
MaxOneClaimPerEpoch();
```

Recommendation

Keep reward settlement on one time basis. The timestamp should move only by the time that was actually settled, while unpaid partial time should remain preserved. The one-claim-per-epoch check is useful, but it should not be the only protection because it only blocks repeated claims inside the same global epoch.

Team Update

We acknowledge this issue and will address it on the frontend to prevent this case from occurring.

TSI - Tokens Sufficiency Insurance

Criticality	Minor / Informative
Location	MagnummetaValidator.sol
Status	Acknowledged

Description

The MagnumMetaValidator contract credits each new validator and delegator position with a principal stake denominated in MFORGE and computed from the full USD value of the user's USDT deposit, while only the fifteen-percent buy portion of that deposit is actually swapped into MFORGE on-chain and the remaining eighty-five percent is forwarded to the devops and affiliate addresses as USDT. Although V2 introduced totalActiveStakeMforge as a running aggregate, this variable is populated from the oracle-derived full-investment figure rather than actual swap proceeds, so it measures an overstated liability and does not correct the underlying shortfall. The withdrawMforgeTokens guard does not account for accrued but unclaimed reward liabilities, leaving the portion of the balance that backs future claims and unstakes unprotected. As a result, the protocol's solvency depends on the owner continuously topping up reserves through depositMforgeTokens, and in scenarios where reserves are insufficient, fee distributions may consume the remaining balance before the final user transfer executes, causing that transfer to revert while leaving the position still marked active with no MFORGE received.

Recommendation

It is recommended to derive the user's credited stake from the MFORGE actually received by the contract rather than from the oracle-quoted full investment value. The team should also consider introducing on-chain liability accounting that tracks both active stake and accrued but unsettled reward obligations as a combined aggregate, and updating the withdrawMforgeTokens guard to verify that the remaining balance covers this full liability before any withdrawal is permitted, so that insolvency surfaces with a clear descriptive error early in the transaction rather than mid-way through an ERC-20 transfer. Additionally,

enforcing a minimum bound on buyPercent would prevent configurations where no MFORGE is purchased while full-value stake is still credited.

Team Update

No code change required. Working as designed.

TTSR - Transfer-Tax Token Swap Risk

Criticality	Minor / Informative
Location	MagnummetaValidator.sol#L1063
Status	Acknowledged

Description

The contract has migrated to `swapExactTokensForTokensSupportingFeeOnTransferTokens`, which correctly handles fee-on-transfer tokens on the input leg. However, no balance-before and balance-after check is performed around the swap to measure the MFORGE actually received by the contract. The `amountOutMin` guard is derived from the oracle price rather than from the real output, and the stake credit is computed from the oracle-quoted full investment value rather than the measured delta. If MFORGE applies a transfer fee or rebase mechanic, the shortfall is silently absorbed by the protocol reserve while the user is still credited the full nominal stake amount, compounding the existing solvency drift described under TSI.

```
uniswapRouter.swapExactTokensForTokensSupportingFeeOnTransferTokens(  
    buyAmount,  
    amountOutMin,  
    path,  
    address(this),  
    block.timestamp + 3600  
);
```

Recommendation

It is recommended to snapshot the contract's MFORGE balance immediately before and after the swap and treat the measured difference as the authoritative amount of MFORGE acquired. Both the slippage guard and the stake-credit calculation should be derived from this measured delta rather than from the oracle quote, ensuring that any transfer fee or rebase on the output token is accurately reflected and that the credited principal cannot exceed the MFORGE actually received by the contract.

Team Update

No code change required. Working as designed.

UPS - Untrusted Pricing Source

Criticality	Medium
Location	MagnummetaValidator.sol#L1052
Status	Acknowledged

Description

`MagnummetaValidator.sol` relies on an external `priceFeed` contract to determine the MFORGE/USD price used for stake sizing and swap slippage checks, but the security properties of this oracle are not enforced in the validator itself. If the configured oracle is inaccurate, stale, or manipulable, the contract can misprice deposits and over-credit users with MFORGE-denominated stake. In particular, an artificially low reported MFORGE price increases `totalStakeGross`, allowing a depositor to receive more stake and rewards than intended.

Because this price directly influences how much stake is minted from a USDT deposit, oracle manipulation can create protocol losses even if the swap itself executes successfully. The contract therefore inherits substantial trust and integrity assumptions from the external price source.

```
interface IPriceFeed {
    function getMforgePriceInUSD(address pancakePair) external view
    returns (uint256);
}

uint256 currentPriceForSlippage =
priceFeed.getMforgePriceInUSD(pancakePair);
...
uint256 currentMforgePriceInUSD =
priceFeed.getMforgePriceInUSD(pancakePair);
uint256 totalStakeGross = (amount * 10**18) / currentMforgePriceInUSD;
```

Recommendation

Consider using a pricing source with well-understood security properties and validating that its output is resistant to manipulation, stale data, and abnormal deviations. It may also be beneficial to add defensive checks around price usage, such as sanity bounds, freshness

validation, and fallback behavior, so the protocol is less exposed to failures or manipulation in a single oracle dependency.

Team Update

No code change required. Working as designed.

UBAF - Upgrade Bypasses APR Floor

Criticality	Minor / Informative
Location	MagnumMetaValidator.sol#L937,996
Status	Unresolved

Description

Latest contracts adds a phase check before allowing a delegator to upgrade:

```
if (livePhase > 1) revert UpgradeOnlyInPhaseOne();
```

This blocks delegators who have already moved into phase 2 or later, so the worst version of the old issue is reduced.

However, the fix does not preserve how far the delegator has already progressed inside phase 1. Phase 1 lasts roughly six months. A user can spend most of that time as a delegator, build up most of their phase-1 reward progress, and then upgrade shortly before entering phase 2. After the upgrade, the contract recalculates the validator phase using the new validator bucket size:

```
uint256 calculatedBucketSize = (finalTotalStake * tier.baseAPR) /  
1000000 / 2;  
uint256 currentPhase = (delegator.rewardsEarned / calculatedBucketSize)  
+ 1;
```

Because the validator stake and bucket are larger, the user's existing delegator reward progress becomes much smaller relative to the new bucket. In practice, the user can be almost finished with delegator phase 1, upgrade, and start again near the beginning of validator phase 1. This gives them extra time at the highest APR compared with preserving their original progress.

Recommendation

The upgrade should carry over the users reward progress fairly, not just check whether they are still in phase 1. If a delegator is already close to moving into the next APR phase, the

upgraded validator position should reflect that progress instead of giving them a near-fresh phase 1 window again.

Team Update

We acknowledge this issue and will address it on the frontend to prevent this case from occurring.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	MagnummetaValidator.sol MagnumMetaValidatorNFT.sol MagnumMetaAffiliate.sol
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function updateTierMetadata(uint256 tier, string memory _name, string
memory _rarity, string memory _investment) external onlyOwner

function setTierPhaseImagesBatch(uint256 tier, string[] calldata
_ipfsLinks) external onlyOwner

function setSinglePhaseImage(uint256 tier, uint256 phase, string memory
_ipfsLink) external onlyOwner

function setValidatorContract(address _validatorContract) external
onlyOwner
```

```
function setValidatorContract(address _validatorContract) external
onlyOwner

function withdrawFallbackBalance() external onlyOwner

function withdrawTokens(address token, uint256 amount) external onlyOwner

function updateReferrerByAdmin(address user, address newReferrer)
external onlyOwner
```

```
function emergencyRevokePosition(address user, uint256 positionId,
bool isValidator) external onlyOwner

function migrateOffChainValidator(
    address user,
    uint256 tierIndex,
    uint256 principalAmount,
    uint256 bonusAmount,
    uint256 originalStartTime,
    address referrer
) external onlyOwner

function migrateOffChainDelegator(
    address user,
    uint256 originalUsdtInvestment,
    uint256 principalAmount,
    uint256 bonusAmount,
    uint256 originalStartTime,
    address referrer
) external onlyOwner

function withdrawMforgeTokens(uint256 amount) external onlyOwner

function updateCoreContracts(address _uniswapRouter, address
_nftContract, address _usdtToken, address _mforgeToken ) external
onlyOwner

function updateWallets(address _devopsWallet, address
_treasuryWallet, address _communityPoolWallet, address
_validatorBonusPoolWallet, address _designatedBurnWallet) external
onlyOwner

function setOracleContracts(address _priceFeed, address _pancakePair)
external onlyOwner

function setSplitPercentages(uint256 _buy, uint256 _affiliate,
uint256 _devops) external onlyOwner

function setAffiliateContract(IAffiliate _affiliateContract)
external onlyOwner

function updateFinancialThresholds(
    uint256 _autoCompoundThreshold,
    uint256 _minWithdrawalAmount
) external onlyOwner

function setSlippageTolerance(uint256 _newSlippage) external
onlyOwner
```

```
function adjustTierSeats(uint256 t1Seats, uint256 t2Seats, uint256  
t3Seats, uint256 t4Seats) external onlyOwner
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

Team Update

No immediate code change required. Centralisation is a known and accepted trade-off at current protocol stage with a clear decentralisation roadmap.

EWRE - Epoch Window Restricts Exits

Criticality	Minor / Informative
Location	MagnumMetaValidator.sol
Status	Acknowledged

Description

The `checkClaimWindow` modifier enforces a 12-hour freeze at the end of every 7-day epoch to prevent accounting inconsistencies during epoch rollover. However, the modifier is applied uniformly to all user-facing functions — including `unstake`, `transferValidator`, and `purchaseValidator` — which have no dependency on epoch boundary accounting. As a result, users with fully matured positions that have completed the 5-year lock period are temporarily prevented from unstaking and recovering their principal for up to 12 hours every week, purely due to the epoch freeze. This compounds with the `whenNotPaused` modifier, meaning users face two independent mechanisms that can simultaneously block exits with no on-chain guarantee of when access resumes. For a protocol with a 5-year lock period, imposing a recurring weekly exit restriction on matured positions is an unnecessary and disproportionate constraint.

```
modifier checkClaimWindow() {  
    if (!isClaimWindow()) revert SystemFrozen();  
    _;  
}  
  
function purchaseValidator(...) external nonReentrant whenNotPaused  
checkClaimWindow  
function stakeDelegator(...) external nonReentrant whenNotPaused  
checkClaimWindow  
function claimRewards(...) external nonReentrant whenNotPaused  
checkClaimWindow  
function transferValidator(...) external nonReentrant whenNotPaused  
checkClaimWindow  
function unstake(...) external nonReentrant whenNotPaused  
checkClaimWindow  
function upgradeDelegatorToValidator(...) external nonReentrant  
whenNotPaused checkClaimWindow
```

Recommendation

It is recommended to narrow the scope of `checkClaimWindow` to only the functions where epoch-boundary accounting is genuinely relevant, specifically reward claiming and compounding. Functions such as `unstake`, `transferValidator`, and `purchaseValidator` do not depend on epoch-aligned reward calculations and should remain accessible throughout the full epoch including the freeze window. Separating the freeze scope by function type would eliminate the recurring exit restriction on matured positions while preserving the accounting integrity the freeze window was designed to protect.

Team Update

Finding acknowledged. No code change required.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	MagnumMetaValidatorNFT.sol#L69,87 MagnumMetaValidator.sol#L1103,1090
Status	Acknowledged

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

```
function _processCompoundFee(uint256 amount) internal returns (uint256  
netAmount)  
    function _distributeFees(uint256 amount) internal  
  
function mintValidatorNFT(address to, uint256 tier) external  
onlyValidator returns (uint256)  
    function burnValidatorNFT(uint256 tokenId) external onlyValidator
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

Team Update

All significant actions are already covered by existing events. No additional event emissions required. Finding closed.

MPV - Missing Parameter Validation

Criticality	Minor / Informative
Location	MagnumMetaValidatorNFT.sol MagnumMetaValidator.sol
Status	Acknowledged

Description

Several privileged configuration functions in `MagnummetaValidator.sol` update core protocol parameters without enforcing meaningful bounds or consistency checks. While some setters validate only non-zero addresses or simple arithmetic totals, others allow arbitrary values for tier economics, lock timing, APR settings, and time-scaling parameters. This creates a configuration risk where an incorrect or careless admin action can materially alter reward behavior, lock mechanics, or tier availability in ways that may break assumptions elsewhere in the system.

Because these parameters directly influence staking eligibility, reward accrual, maturity, and economic splits, insufficient validation can lead to accidental misconfiguration with user-facing consequences even in the absence of malicious intent. This is especially relevant for functions that govern time and APR behavior, where extreme values may distort reward calculations or make the protocol operate outside its intended design envelope.

The function `setSplitPercentages` only validates that the total percentage equals 100, but does not enforce a minimum allocation for each component. This allows values such as `_buy = 0` or `_affiliate = 0`, potentially disabling important fee distribution paths.

```
require(_buy + _affiliate + _devops == 100, "must be 100");
```

The function `setSlippageTolerance` enforces only an upper limit of 15% slippage but does not validate a minimum value. Allowing zero slippage may cause all swaps to fail due to normal price movement during execution.

```
if (_newSlippage > 1500) revert Max15Percent();
```

The function `adjustTierSeats` validates only that the total number of seats equals 250, without enforcing a minimum allocation per tier. This allows critical tiers such as `t1Seats` to be set to zero while still passing validation.

```
if (t1Seats + t2Seats + t3Seats + t4Seats != 250) revert  
Exact250Seats();
```

The function `updateTierMetadata` does not validate whether the provided `tier` value is within the valid tier range. As a result, metadata may be assigned to non-existent or unintended tiers.

```
tierMetadata[tier] = TierMetadata(_name, _rarity, _investment);
```

The function `setTierPhaseImagesBatch` validates only the number of IPFS links provided but does not verify that the specified tier actually exists. This may result in image data being assigned to invalid tiers.

```
require(_ipfsLinks.length == 5, "Must provide exactly 5 IPFS links for  
Phase 1 to 5");
```

Recommendation

Consider applying explicit validation rules to all governance-controlled parameters that affect protocol economics, timing, and tier configuration. In general, bounded ranges, relationship checks between related values, and invariant enforcement can help reduce the risk of accidental misconfiguration and keep administrative updates aligned with the intended operating model.

Team Update

Split percentage flexibility is intentional — protocol reserves the right to pause affiliate or DEX buy programs independently. Total = 100 is the only enforced invariant. Remaining three guards for slippage floor, tier seats minimum, and tier range validation will be added. Finding partially acknowledged. No change.

PF - Pausable Functionality

Criticality	Minor / Informative
Location	MagnummetaValidator.sol MagnumMetaAffiliate.sol
Status	Acknowledged

Description

Both `MagnummetaValidator.sol` and `MagnumMetaAffiliate.sol` implement pausable logic over core user flows, allowing privileged accounts to halt major protocol operations. In the validator, staking, claiming, transfers, unstaking, and upgrades are gated by `whenNotPaused`, and pausing can be triggered not only by the owner but also by any address marked as authorized. In the affiliate contract, reward distribution and affiliate claiming are similarly stoppable by the owner. This creates a strong administrative trust assumption, as privileged actors can freeze normal user activity at will.

While emergency stops can be useful for incident response, they also introduce centralization risk and operational dependency on the admin set. Users are exposed to the possibility that deposits remain locked in inactive positions or affiliate rewards become temporarily inaccessible whenever the pause mechanism is exercised.

```
function pause() external onlyOwner { _pause(); }
function unpause() external onlyOwner { _unpause(); }

function purchaseValidator(...) external nonReentrant whenNotPaused
checkClaimWindow
function stakeDelegator(...) external nonReentrant whenNotPaused
checkClaimWindow
function claimRewards(...) external nonReentrant whenNotPaused
checkClaimWindow
function transferValidator(...) external nonReentrant whenNotPaused
checkClaimWindow
function unstake(...) external nonReentrant whenNotPaused
checkClaimWindow
function upgradeDelegatorToValidator(...) external nonReentrant
whenNotPaused checkClaimWindow

function pause() external onlyOwner { _pause(); }
function unpause() external onlyOwner { _unpause(); }
```

```
function distributeAffiliate(address buyer, address referrer, uint256  
affiliateAmount) external onlyValidatorContract whenNotPaused  
function claimAffiliateRewards() external nonReentrant whenNotPaused
```

Recommendation

Consider clearly acknowledging the administrative trust model around pause powers and limiting those privileges to the minimum set of actors necessary for operational safety. It may also be beneficial to add governance, timelock, multi-signature control, or narrowly scoped emergency procedures so the pause mechanism remains available for security incidents without creating unnecessary centralized control over routine user operations.

Team Update

No code change required. Multisig migration resolves the trust assumption. Finding closed upon multisig deployment.

UTPD - Unverified Third Party Dependencies

Criticality	Minor / Informative
Location	MagnummetaValidator.sol#L1037
Status	Acknowledged

Description

The contract uses an external contract in order to determine the transaction's flow. The external contract is untrusted. As a result, it may produce security issues and harm the transactions.

```
function _processInvestmentAndSwap(uint256 amount) internal returns
(uint256 userPrincipalNet, uint256 affiliateAmount) {
    ...
    uint256 currentPriceForSlippage =
priceFeed.getMforgePriceInUSD(pancakePair);
    ...
}
```

Recommendation

The contract should use a trusted external source. A trusted source could be either a commonly recognized or an audited contract. The pointing addresses should not be able to change after the initialization.

Team Update

Finding closed. Third party dependency is audited, verified, and live on mainnet. No code change required.

Functions Analysis

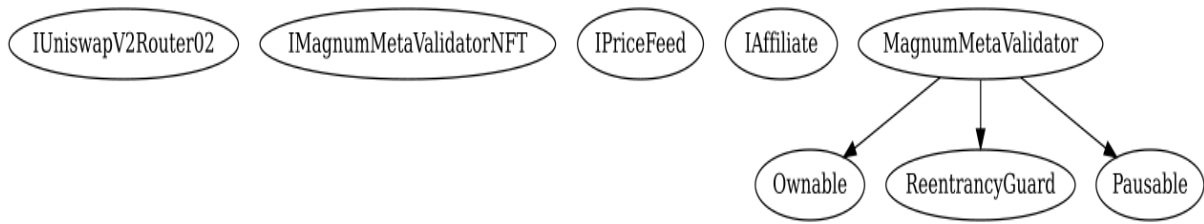
Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
IUniswapV2Router02	Interface			
	swapExactTokensForTokens	External	✓	-
	swapExactTokensForTokensSupportingFeeOnTransferTokens	External	✓	-
IMagnumMetaValidatorNFT	Interface			
	mintValidatorNFT	External	✓	-
	burnValidatorNFT	External	✓	-
	transferFrom	External	✓	-
	ownerOf	External		-
	balanceOf	External		-
	tokenOfOwnerByIndex	External		-
IPriceFeed	Interface			
	getMforgePriceInUSD	External		-
IAffiliate	Interface			
	distributeAffiliate	External	✓	-

	registerLegacyAffiliate	External	✓	-
MagnumMetaValidator	Implementation	Ownable, ReentrancyGuard, Pausable		
		Public	✓	Ownable
	purchaseValidator	External	✓	nonReentrant whenNotPaused checkClaim Window
	stakeDelegator	External	✓	nonReentrant whenNotPaused checkClaim Window
	claimRewards	External	✓	nonReentrant whenNotPaused checkClaim Window
	calculateRewards	Public		-
	powerWAD	Internal		
	getCurrentAPR	Public		-
	getDelegatorAPR	Public		-
	isClaimWindow	Public		-
	getCurrentEpoch	Public		-
	transferValidator	External	✓	nonReentrant whenNotPaused

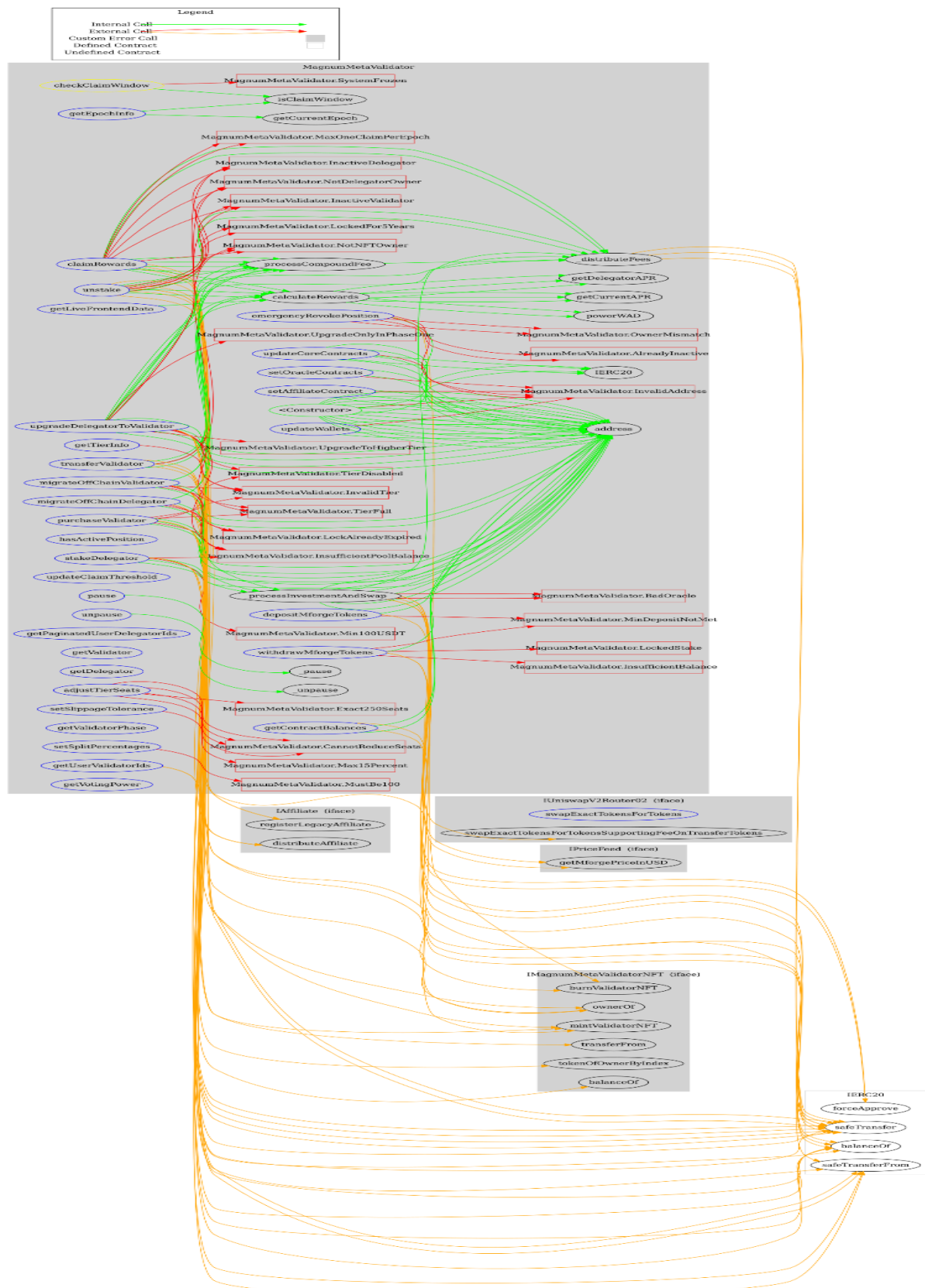
				checkClaimWindow
	unstake	External	✓	nonReentrant whenNotPaused checkClaimWindow
	emergencyRevokePosition	External	✓	onlyOwner
	migrateOffChainValidator	External	✓	onlyOwner
	migrateOffChainDelegator	External	✓	onlyOwner
	upgradeDelegatorToValidator	External	✓	nonReentrant whenNotPaused checkClaimWindow
	pause	External	✓	onlyOwner
	unpause	External	✓	onlyOwner
	_processInvestmentAndSwap	Internal	✓	
	_distributeFees	Internal	✓	
	_processCompoundFee	Internal	✓	
	hasActivePosition	External		-
	setAffiliateContract	External	✓	onlyOwner
	setOracleContracts	External	✓	onlyOwner
	depositMforgeTokens	External	✓	onlyOwner
	withdrawMforgeTokens	External	✓	onlyOwner
	updateWallets	External	✓	onlyOwner
	updateCoreContracts	External	✓	onlyOwner

	adjustTierSeats	External	✓	onlyOwner
	updateClaimThreshold	External	✓	onlyOwner
	setSlippageTolerance	External	✓	onlyOwner
	setSplitPercentages	External	✓	onlyOwner
	getPaginatedUserDelegatorIds	External		-
	getValidator	External		-
	getDelegator	External		-
	getTierInfo	External		-
	getContractBalances	External		-
	getEpochInfo	External		-
	getValidatorPhase	External		-
	getUserValidatorIds	External		-
	getLiveFrontendData	External		-
	getVotingPower	External		-

Inheritance Graph



Flow graph



Summary

Magnummeta contracts implements a staking suite built around a validator/delegator product, an affiliate referral program, and an NFT-based validator identity layer. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

